

Informe de Proyecto StockSmart - Sistema de Gestión de Inventario

Nombre: Alison Hidalgo Guerra

Grupo: 22 **No.Lista:** 27

Asignatura: Optativa I -JavaScript-

Introducción

Este proyecto presenta el desarrollo de una aplicación web denominada StockSmart, un sistema de gestión de inventario diseñado para facilitar el control de productos, categorías y movimientos en entornos comerciales pequeños. La aplicación incluye una interfaz web intuitiva y funcional, desarrollada con tecnologías modernas de JavaScript.

Objetivos del Proyecto

El proyecto busca:

- Crear una aplicación web completa que integre todos los temas requeridos.
- Implementar funcionalidades útiles para la gestión de inventario.
- Asegurar que el código sea claro, modular y bien comentado.

Tecnologías Utilizadas

- **Lenguaje Principal:** JavaScript .
- **Framework Frontend:** React 19.0.1, utilizando hooks como useState, useEffect y useMemo.
- **Herramientas de Desarrollo:**
 - Vite 6.2.3 para el servidor de desarrollo y bundling.
 - Tailwind CSS 4.2.4 para el diseño responsivo.
- **Librerías Adicionales:**
 - Recharts para gráficos.
 - Lucide React para íconos.
 - date-fns para manejo de fechas.
 - clsx y tailwind-merge para clases CSS dinámicas.
- **Entorno:** Node.js , con scripts npm para desarrollo y construcción.

Funcionalidades del Sistema

La aplicación StockSmart ofrece las siguientes características:

Dashboard

- Visualización de métricas clave: stock total, alertas de productos críticos y ganancias en USD/CUP.
- Gráficos interactivos para tendencias de almacén y distribución por categorías.
- Actualización automática cada minuto.

Gestión de Inventario

- Lista de productos con opciones de filtrado (categoría, stock bajo, búsqueda) y ordenamiento (fecha, nombre, código, stock).
- Operaciones por producto: edición, eliminación, registro de ventas y consulta de historial.

Gestión de Categorías

- Creación, edición y eliminación de categorías.
- Validación para prevenir eliminación de categorías con productos asociados.

Estadísticas y Reportes

- Cálculos de ganancias estimadas por producto y globales.
- Identificación de productos con stock bajo.
- Historial completo de movimientos de inventario.

Configuración

- Cambio de idioma (español/inglés).
- Métodos de cálculo de ganancias (simple, fijo, porcentual).
- Configuración de tasa de cambio y umbral de stock bajo.

Interfaz de Usuario

- Diseño responsivo en modo oscuro.
- Modales para interacciones de usuario.
- Navegación por pestañas.

Cumplimiento de los Temas de la Asignatura

A continuación, se detalla el cumplimiento de cada tema con ejemplos de código.

Tema 1: JavaScript como Lenguaje de Programación

Se refleja en validaciones y funciones básicas.

Ejemplo: Función `validateProductCode` en `utils.js` (líneas 10-18):

```
export const validateProductCode = (code) => {  
  let isValid = false;  
  const regex = /^[A-Z]{3}\d{3}$/;  
  if (typeof code === 'string' && code.length === 6) {  
    isValid = regex.test(code);  
  }  
  return isValid;  
};
```

Incluye variables, tipos de datos, estructuras de control y funciones.

Tema 2: Objetos y Programación Orientada a Objetos

Implementado mediante objetos literales y una clase.

Ejemplo: Clase `Product` en `types.js` (líneas 25-45):

```
export class Product {
  constructor(id, code, name, category, basicPrice, sellingPrice, currency, stock)
  {
    this.id = id;
    this.code = code;
    this.name = name;
    this.category = category;
    this.basicPrice = basicPrice;
    this.sellingPrice = sellingPrice;
    this.currency = currency;
    this.stock = stock;
  }

  calculateProfit() {
    return this.sellingPrice - this.basicPrice;
  }

  isLowStock(threshold) {
    return this.stock <= threshold;
  }

  getTotalValue() {
    return this.sellingPrice * this.stock;
  }
}
```

Tema 3: Manejo de Datos y Programación Funcional

Uso extensivo de métodos de arrays y closures.

Ejemplo: Cálculo de ganancias en App.jsx (líneas 912-918):

```
const totalGainUSD = products.reduce((acc, p) => {
  return acc + (p.sellingPrice - p.basicPrice) * p.stock;
}, 0);
```

Closures en `useMemo` (línea ~991).

Tema 4: Modelo de Ejecución y Asincronía

Implementado con callbacks, Promises y async/await.

Ejemplo: Carga asíncrona en App.jsx (líneas 725-737):

```
const loadInitialData = async () => {
  const dataPromise = new Promise((resolve) => {
    setTimeout(() => resolve({ products: INITIAL_PRODUCTS, ... }), 500);
  });
  const data = await dataPromise;
```

```
setProducts(data.products);  
};
```

Callback en `setInterval` (línea ~731).

Estructura del proyecto

Raíz del proyecto

- `index.html`
 - Página base donde se monta la aplicación React.
- `package.json`
 - Dependencias y scripts de npm (`dev` , `build` , `preview` , `lint`).
- `vite.config.js`
 - Configuración de Vite para React y Tailwind.
- `README.md`
 - Documentación del proyecto.
- `metadata.json`
 - Descripción general para el contenedor o exportación.
- `temp_content.txt`
 - Archivo auxiliar con contenido de traducción / textos.
- `.gitignore`
 - Archivos y carpetas ignorados por Git.

Carpeta `src`

- `main.jsx`
 - Punto de entrada de la app React.
 - Renderiza `<App />` dentro de `#root` .
- `App.jsx`
 - Componente principal.
 - Contiene toda la lógica de la app: estado, hooks, modales, filtros, gráficos, CRUD de productos y categorías.
 - Aquí se implementan las funciones relevantes para los temas de JavaScript.
- `index.css`
 - Estilos globales y clases base.
 - Complementa Tailwind para la apariencia general.
- `types.js`
 - Definiciones JSDoc.
 - Incluye la clase `Product` y tipos usados por la app.
- `utils.js`
 - Funciones utilitarias compartidas.
 - Ejemplo: `formatCurrency` y `validateProductCode` .

Carpeta `lib`

- `utils.js`
 - Utilidades genéricas de la aplicación.
 - Este archivo ayuda a separar lógica de formato/validación del componente principal.